SAE INSTITUTE

The Best Parts of C++

Elias Farhan

University of Hertfordshire UH

Which C++ standards do you use?

(and which one does your compiler support?)

- C++98
- C++03
- C++11
- C++14
- C++17
- C++20
- C++23
- C++26

Who decide the C++ standard ?

- A dictator?
- A company?
- Big tech?
- Mad scientists?

WG21 (The Committee)

Convenor (Chair): Guy Davidson since 2025. ~100-200 activate participants.

Main Working Groups

- EWG (Evolution Working Group) - Language evolution proposals
- LEWG (Library Evolution Working Group) - Standard library proposals

Key Principles Guiding Decisions

The committee adheres to several core philosophies:

- Zero Overhead Abstraction: Features should not incur runtime costs if they aren't used.
- Backward Compatibility: The committee strives to keep old code compiling on new standards (though rare breaking changes occur for safety or correctness).
- Pragmatism: Theoretical elegance is often weighed against practical implementation difficulty and real-world utility.
- Safety and Security: Modern C++ evolution places a heavier emphasis on memory safety and reducing undefined behavior compared to earlier versions.

Ok, but what is a standard?

The official, internationally recognized document that defines exactly how the C++ programming language works. So the most recent standard, the more features!

The reference is here: `https://en.cppreference.com/`

The compiler

Of course, the standard means nothing if we can't use it. This is where the compiler comes into play. Those are the three main compilers in use today:

- MSVC (<- you used this one)
- GCC
- Clang

Where does our code run?

C++ compiles to native code... but for which platform?

- Windows (your laptop)
- Linux (servers, Steam Deck)
- macOS
- iOS / Android
- Consoles (PS5, Xbox, Switch)
- Web (via Emscripten)

SAE INSTITUTE

The catch

Each platform has different:

- File paths (/ vs \ , `/home/dev/cpp_project` vs `c:\Users\unite\Dev\cpp_project`)
- System APIs (Win32 vs POSIX)
- Compilers (MSVC, GCC, Clang)
- Project files (`.sln`, `.xcodeproj`, `Makefile`...)

How do we build a C++ project?

```
// main.cpp + 50 other .cpp files
```

- Windows? Visual Studio `.sln`
- macOS? Xcode `.xcodeproj`
- Linux? `Makefile`
- All platforms? ...

What if there was a way...

to describe the build *once*, and generate any project format?

CMake

Cross-platform build system generator. One `CMakeLists.txt` → any compiler / IDE.

```
cmake_minimum_required(VERSION 3.20)
project(my_game CXX)
set(CMAKE_CXX_STANDARD 23)
add_executable(my_game main.cpp player.cpp)
```

How do we manage code over time?

- “It worked yesterday...”
- “Whose version is the latest?”
- `my_project_FINAL_v2_real_final.zip`

What if we could track every change, every author, every version?

Git

Distributed version control.

- Every commit is a full snapshot
- Branches let you experiment safely
- Merge contributions from many developers
- Industry standard since ~2010

How do we use libraries?

We need: JSON parsing, networking, GUI, audio, physics...

C++ has no built-in package manager.

Bad answer: download, build manually, copy DLLs, fix include paths, pray.

vcpkg

C++ package manager (Microsoft, open-source, cross-platform).

```
{
    "dependencies": [
        "fmt",
        "nlohmann-json",
        "sdl2"
    ]
}
```

WTF, Windows?

C++ on Windows is... a journey.

- **Visual Studio**: full IDE, also installs MSVC
- **VS Build Tools**: just the compiler, no IDE
- **CLion / VS Code**: alternative editors, still need MSVC installed
- **CMake**: ties them all together

Whose brace style?

```
if (x) {          // K&R?
    foo();
}

if (x)            // Allman?
{
    foo();
}
```

clang-format

A `.clang-format` file at the root → auto-format on save / commit.

Predefined styles: `Google`, `LLVM`, `Mozilla`, `Microsoft`... Or define your own.

Code style stops being an opinion. It becomes a config file.

Where do declarations go?

```
// math_func.h  -- header: declarations
double area_of_circle(double radius);

// math.cpp -- source: definitions
#include "math_func.h"
double area_of_circle(double radius) {
    return 3.141593 * radius * radius;
}
// main.cpp -- consumer
#include "math_func.h"
int main() { return area_of_circle(1.5); }
```

Compilation + Linking

- Each `.cpp` is compiled **independently** into a `.o`
- The **linker** stitches the `.o` files together with libraries
- Headers tell the compiler what *exists*; sources provide the implementation

```
math.cpp ──[compiler]──> math.o   ┐
main.cpp ──[compiler]──> main.o   ├──[linker]──> my_app
                         libfmt   ┘
```

What does the standard actually describe?

Not your CPU. Not your OS.

A **C++ abstract machine**: a hypothetical computer with strict rules about how C++ programs behave. Continuous memory, executing instructions one after the other...

The compiler is free to optimize as wildly as it wants...

...as long as observable behavior matches.

This is the **“as-if” rule**.

```
int sum = 0;
for (int i = 0; i < 1000; ++i) sum += i;
std::cout << sum;
```

A modern optimizer turns this into:

```
std::cout << 499500;
```

What does `std::` even mean?

What's that prefix for? Imagine two libraries — physics and graphics — both with their own `Vector`:

```
// physics.h
struct Vector { float x, y, z; };

// graphics.h
struct Vector { float x, y; };

#include "physics.h"
#include "graphics.h"

Vector v;  // which one?!
```

Namespaces

A namespace is a named scope. Names defined inside it are qualified with `namespace::name`. `std::` is the namespace of the standard library.

```
namespace phys { struct Vector { float x, y, z; }; }
namespace gfx  { struct Vector { float x, y;    }; }

phys::Vector world_pos;
gfx::Vector  screen_pos;

using gfx::Vector;     // surgical: only Vector
using namespace gfx;   // wholesale: everything from gfx (avoid in headers)
```

Value, reference, pointer

```
struct Player { /* big struct */ };

void by_value(Player p);
void by_ref  (Player& p);
void by_ptr  (Player* p);

Player hero;
by_value(hero);
by_ref(hero);
by_ptr(&hero);
by_ptr(nullptr);
```

Value, reference, pointer

```
void by_value(Player p);       // copies the whole Player
void by_ref  (Player& p);      // alias to caller's variable
void by_ptr  (Player* p);      // address; may be null, may be re

Player hero;
by_value(hero);    // copy made
by_ref(hero);      // no copy, hero can be modified
by_ptr(&hero);     // no copy, address taken explicitly
by_ptr(nullptr);   // valid: pointer can be null
```

When to use which?

- **Value** — small / cheap types (`int`, `float`, `Vec3`), or when you genuinely want a copy.
- `const T&` — read-only access to something larger than a register. The default for “pass me a thing”.
- `T&` — when the function needs to *modify* the caller's variable.
- `T*` — when the argument is genuinely optional (`nullptr` means “no value”), or when ownership / array semantics are involved.

Let's start with some beginner code

Is it possible for `pi` to change?

```
#include < iostream >
double pi = 3.141593;
int main()
{
    double area, radius = 1.5;
    area = pi * radius * radius;
    std::cout << area;
}
```

What if there was a way...

to specify that the value for `pi` cannot change...

```
#include < iostream >
double pi = 3.141593;
int main()
{
    double area, radius = 1.5;
    area = pi * radius * radius;
    std::cout << area;
}
```

const

const tells the compiler that a value cannot change

```
#include < iostream >
const double pi = 3.141593;
int main()
{
    double area, radius = 1.5;
    area = pi * radius * radius;
    std::cout << area;
}
```

const

If we apply `const` consistently we simplify our variable declarations

```
#include < iostream >
const double pi = 3.141593;
int main()
{
    const double radius = 1.5;
    const double area = pi * radius * radius;
    std::cout << area;
}
```

East const vs west const

`const` applies to the element on its left (or right if there is nothing on the left)

```
const int i = 5; /// must be initialized, cannot change
int const j = 6; // or this way around
const int& ref = j; //int is const, not the reference
const char* text = "Hello world!"; // char is const, not the *
```

const member methods

A method tagged `const` promises **not to modify** the object.

```
class Player {
  public:
    int  score() const { return score_; }  // const method
    void AddPoint()       { ++score_; }       // non-const
  private:
    int score_ = 0;
};

const Player& hero;
hero.score();   // OK
hero.AddPoint();   // compile error: hero is const
```

A player state with a plain enum

Old `enum` has two problems: names leak into the surrounding scope, and values convert silently to `int`.

```
enum State  { Idle, Running, Jumping };
enum Weapon { Sword, Bow, Idle };  // collision: Idle already t

State s = Running;
if (s == 1)     { /* compiles silently */ }
if (s == Sword) { /* also compiles! */ }
```

enum class

C++11 introduced **scoped, strongly-typed** enums.

```
enum class State  { Idle, Running, Jumping };
enum class Weapon { Sword, Bow, Idle };  // OK, separate scope

State s = State::Running;
if (s == 1)             { /* compile error */ }
if (s == Weapon::Idle) { /* compile error */ }
if (s == State::Idle)  { /* OK */ }
```

A `Player` with no rules

Anyone can put this object in a nonsense state. There's no enforcement.

```
struct Player {
    std::string name;
    int score;
    int health;
};

Player hero;
hero.score  = -42;       // fine
hero.health = 9999999;   // fine
hero.name   = "";        // fine!
```

Constructors

A constructor runs when an object is created. It can initialize fields and enforce invariants.

```
class Player {
public:
    Player(const std::string& n, int h)
        : name_{n}, score_{0}, health_{h}
    {}

private:
    std::string name_;
    int score_;
    int health_;
};

Player hero{"Alice", 100};
hero.health_ = 9999999;  // compile error: private!
```

`class` vs `struct`, and `explicit`

`class` is just `struct` with `private` as the default access. A single-argument constructor enables **implicit conversion** unless marked `explicit`.

```
struct Point { int x; int y; };        // public by default
class  Vec3  { float x_, y_, z_; };    // private by default
```

```
class Player {
public:
    explicit Player(int health) : health_{health} {}
};

void start_game(Player p);

start_game(100);         // error: must be Player{100}
start_game(Player{100}); // OK
```

Comparing `Players`

We want to sort players by score, deduplicate them in a `std::set`, look them up in a `std::map`. The type needs $<$, $\leq$, $>$, $\geq$, $=$, $\neq$ — six operators, all derived from the same comparison.

```
struct Player {
    std::string name;
    int score;
};

bool operator< (const Player& a, const Player& b) { return a.score <  b.score; }
bool operator=(const Player& a, const Player& b) { return a.score = b.score; }
bool operator≤(const Player& a, const Player& b) { return a.score ≤ b.score; }
bool operator≥(const Player& a, const Player& b) { return a.score ≥ b.score; }
bool operator> (const Player& a, const Player& b) { return a.score >  b.score; }
bool operator≠(const Player& a, const Player& b) { return a.score ≠ b.score; }
```

Three-way comparison ⟺ (C++20)

One operator generates them all. = `default` means: compare member-by-member, in declaration order.

```
struct Player {
    std::string name;
    int score;

    auto operator⟺(const Player& other) const { return score ⟺ other.score; }
};

Player a{"Alice", 100}, b{"Bob", 80};

a < b;   // false
a == b;  // false
std::sort(players.begin(), players.end());  // works
```

Let's have several values

What if there was a way to contain several values...

```
int p1_score = 0;
int p2_score = 0;
int p3_score = 0;
int p4_score = 0;
```

Bad answers

- `std::map`
- `std::list`

Containers!

- `std::array`: when we know the maximum amount
- `std::vector`: when we don't know the maximum amount or when we need a LOT of values
- `std::unordered_map`: when we don't order things with indices.
- What else?

New additions in C++26

- `std::hive`: when iterator needs to stay valid with growth (not the case with `std::vector`)
- `std::inplace_vector`: when we know the maximum amount, but we want the same behavior than `std::vector`

Let's have several values - fixed!

```
std::array< int, 4 > player_scores{};
```

Looking up values

Has a player unlocked an achievement?

```
std::vector< std::string > unlocked;
// O(n) every call. With thousands of achievements... ouch.
bool has_unlocked(std::string_view name) {
    for (const auto& a : unlocked)
        if (a == name) return true;
    return false;
}
```

What if there was a way...

to ask “do I have this?” without iterating the whole container?

std::set / std::unordered_set

```
std::unordered_set< std::string > unlocked;

bool has_unlocked(std::string_view name) {
    return unlocked.contains(name);
}
```

- `std::set`: ordered, O(log n)
- `std::unordered_set`: hashed, O(1) average
- `std::flat_set` (C++23): sorted vector, cache-friendly for small N

Why “flat”?

```
std::map< K, V >          : tree of nodes,    pointers everywhere
std::unordered_map< K, V >     : buckets of nodes, pointers everywhere
std::flat_map< K, V >     : two contiguous arrays
```

Pointer-chasing kills the cache. Flat containers stay in cache.

Trade-off: slower mid-insertion, faster lookup, smaller memory footprint.

How do we walk through a container?

```
std::vector< int> scores{42, 17, 99, 8};
std::array< int, 4 > ranks{1, 2, 3, 4};
std::unordered_set< int > ids{10, 20, 30};
```

Different containers. Is there a unifying interface?

begin() / end()

Every standard container provides them.

```
for (auto it = scores.begin(); it ≠ scores.end(); ++it) {
    std::cout << *it << '\n';
}
```

Range-based `for` is just sugar for the same thing:

```
for (const auto& s : scores) std::cout << s << '\n';
```

Passing containers around

I want a function that prints a range of `int`s.

```
void print(const std::vector< int >& v);
void print(const std::array< int, 10 >& a);
void print(const int* p, std::size_t n);
```

Three overloads for the same logic?

std::span (C++20)

A non-owning view over a contiguous range.

```
void print(std::span< const int > values) {
    for (int v : values) std::cout << v << ' ';
}

print(my_vector);
print(my_array);
print({1, 2, 3, 4});
```

Random for any number type

We need random numbers — but the type matters. This is how it would look like in C.

```c
int    random_value_int   (int min, int max);
float  random_value_float (float min, float max);
double random_value_double(double min, double max);

auto i = random_value_int   (0, 10);
auto f = random_value_float (0.0f, 1.0f);
auto d = random_value_double(0.0, 1.0);
```

Function overloading

C++ lets multiple functions share a name, as long as their **parameter types differ**.

```
int    random_value(int min, int max);
float  random_value(float min, float max);
double random_value(double min, double max);

random_value(0, 10);        // calls int version
random_value(0.0f, 1.0f);   // calls float version
random_value(0.0, 1.0);     // calls double version
```

What does NULL actually pass?

In old C++, NULL was just 0 — an integer, not a pointer. Combined with function overloading, this gives silent bugs.

```
void process(int);
void process(int*);

process(NULL);   // calls process(int) - silent bug
process(0);      // same problem
```

nullptr (C++11)

`nullptr` has its own type, `std::nullptr_t`, that converts to any pointer but **not** to integers.

```
void process(int);
void process(int*);

process(nullptr);   // calls process(int*), unambiguous

int* p = nullptr;   // OK
int  i = nullptr;   // compile error
```

What about generic code?

```
int    random_value(int min, int max);
float  random_value(float min, float max);
double random_value(double min, double max);
```

Same name now — but still **three implementation copies**. Add a new type → another copy.

What if there was a way...

to write the function *once* and have the compiler stamp out a version for each type?

Templates

```
template< typename T >
T random_value(T min, T max);

int   i = random_value(0, 10);
float f = random_value(0.0f, 1.0f);
```

The compiler instantiates a separate version for each type used.

But it accepts *any* type…

```
random_value(std::string("a"), std::string("z")); // ???
```

The error message? Three pages of template gibberish, deep in the standard library.

What if the function could *say* what types it accepts?

Concepts (C++20)

```
template< std::integral T >
T random_value(T min, T max);

template< std::floating_point T >
T random_value(T min, T max);

random_value(0, 10);          // OK, integer overload
random_value(0.0f, 1.0f);     // OK, floating overload
random_value("a", "z");       // clear error: not integral or flo
```

One template, type-dependent body

We want `to_string` for any type — but the implementation depends on the type. A regular `if` won't do: every branch must compile for **every** `T`, so the string branch fails when `T = int`.

```cpp
template< typename T >
std::string to_string(T value) {
    if (std::is_integral_v< T >)
        return std::to_string(value);
    else if (std::is_floating_point_v< T >)
        return std::to_string(value);
    else
        return std::string{value};  // breaks when T is int
}
```

`if constexpr` (C++17)

Branch evaluated at compile time. The non-taken branch isn't even instantiated.

```cpp
template< typename T >
std::string to_string(T value) {
    if constexpr (std::is_integral_v< T > ||
                  std::is_floating_point_v< T >) {
        return std::to_string(value);
    } else {
        return std::string{value};
    }
}

to_string(42);        // only the numeric branch is instantiated
to_string("hello");   // only the string branch is instantiated
```

A clamp function

x is constant... but it's still computed at runtime. Why?

```
int clamp(int v, int lo, int hi) {
    return v < lo ? lo : v > hi ? hi : v;
}

const int x = clamp(42, 0, 100);
```

What if there was a way...

to compute the value at *compile time*?

constexpr

```
constexpr int clamp(int v, int lo, int hi) {
    return v < lo ? lo : v > hi ? hi : v;
}

constexpr int x = clamp(42, 0, 100); // computed by the compiler
```

Zero runtime cost. The value is baked into the binary.

`constexpr` is implicitly `inline`

Which means you can define it in a **header**, include it in 50 files, and the linker won't complain about duplicate definitions.

```
// math.h
constexpr double square(double x) { return x * x; }
```

No `.cpp` file needed for this function.

constexpr methods

Whole types can be `constexpr`.

```
struct Vec3 {
    float x, y, z;
    constexpr float length_sq() const {
        return x*x + y*y + z*z;
    }
};

constexpr Vec3 v{1, 2, 2};
static_assert(v.length_sq() == 9.0f); // checked at compile time!
```

What's the type of this?

```
std::map< std::string, std::vector< int > > data;
std::map< std::string, std::vector< int > >::iterator it = data.begin();
```

Type names get long. And they change when you change the container.

auto

```
auto it = data.begin();
```

The compiler knows the type. Let it figure it out.

Use it for: iterators, lambdas, range-`for` variables, complex template results.

But auto

```
TextureManager& GetTextureManager();

auto texture_manager = GetTextureManager();
```

Is auto of type TextureManager or TextureManager&?

auto& !!!

```
TextureManager& GetTextureManager();

auto& texture_manager = GetTextureManager();
```

SAE INSTITUTE

Return type deduction

```
auto add(int a, int b)       { return a + b; } // returns int
auto add(double a, double b) { return a + b; } // returns double
```

Especially useful in templates where the return type depends on the parameters:

```
template< typename A, typename B >
auto multiply(A a, B b) { return a * b; }
```

Iterating a map

```cpp
std::unordered_map< std::string, int > scores{
    {"Alice", 100}, {"Bob", 80}, {"Carol", 95}
};

for (const auto& entry : scores) {
    std::cout << entry.first << ": " << entry.second << '\n';
}
```

entry.first and entry.second? Hard to read at a glance.

Structured bindings (C++17)

The compiler unpacks the `std::pair` into named variables.

```
for (const auto& [name, score] : scores) {
    std::cout << name << ": " << score << '\n';
}
```

Structured bindings (C++17)

Works on anything “destructurable”:

```
struct Point { int x, y, z; };
Point p{1, 2, 3};
auto [x, y, z] = p;         // aggregates / structs

std::tuple< int, std::string > t{42, "hi"};
auto [n, s] = t;            // tuples
```

A leaking iterator

`it` only matters inside the `if` — but it lives until the end of the function, ready to be misused 50 lines later.

```
auto it = scores.find("Alice");
if (it != scores.end()) {
    std::cout << it->second << '\n';
}

// `it` is still in scope here.
```

`if` / `switch` with init statement (C++17)

Declare the variable as part of the `if`. It's scoped to the branches only.

```cpp
if (auto it = scores.find("Alice"); it != scores.end()) {
    std::cout << it->second << '\n';
}
// `it` no longer exists here.

switch (auto state = compute_state(); state) {
    case State::Idle:    /* ... */ break;
    case State::Running: /* ... */ break;
}
```

Which one is x?

Position-based aggregate initialization is fragile: reorder the struct, and every call site silently means something different.

```
struct Vec3 { float x, y, z; };
Vec3 v{1.0f, 2.0f, 3.0f};

// later, somebody refactors:
struct Vec3 { float z, x, y; };
Vec3 v{1.0f, 2.0f, 3.0f};  // silently means {z=1, x=2, y=3}
```

Designated initializers (C++20)

Name the field at the call site. Self-documenting, compile-checked, order-checked.

```
struct Vec3 { float x, y, z; };

Vec3 v{.x = 1.0f, .y = 2.0f, .z = 3.0f};

Vec3 origin{};                      // all zero-initialized
Vec3 up{.y = 1.0f};                 // others zero-initialized
Vec3 wrong{.y = 1.0f, .x = 2.0f};   // error: out of order
```

Find the first big score

```
std::vector< int > scores{3, 17, 8, 42, 9, 67};

auto found = scores.end();
for (auto it = scores.begin(); it ≠ scores.end(); ++it) {
    if (*it > 50) { found = it; break; }
}
```

Logic + iteration intertwined. Easy to get wrong.

What if there was a way...

to pass the *condition* itself as a parameter?

Lambdas + <algorithm>

```
auto found = std::ranges::find_if(scores,
    [](int s) { return s > 50; });
```

- `[](int s) { ... }` is a **lambda**: an anonymous function
- `find_if` is one of dozens of standard algorithms
- The compiler usually inlines the lambda — zero overhead

Other useful ones: `sort`, `count_if`, `transform`, `accumulate`, `for_each`.

SAE INSTITUTE

What if the threshold is a variable?

The lambda doesn't compile — `threshold` isn't visible inside its body.

The square brackets `[ ]` aren't decoration: they're the **capture list**.

```
std::vector< int > scores{3, 17, 8, 42, 9, 67};
int threshold = 50;

auto found = std::ranges::find_if(scores,
    [](int s) { return s > threshold; });  // error: threshold
```

Lambda captures

Put the variable in the brackets. By value: the lambda gets its own copy. By reference: the lambda sees the live variable.

```
int threshold = 50;

auto by_value = [threshold](int s)  { return s > threshold; };
auto by_ref   = [&threshold](int s) { return s > threshold; };

threshold = 100;
by_value(60);   // false (sees the captured copy: 50)
by_ref(60);     // true  (sees the live variable: 100)
```

“Maybe a value, maybe nothing”

We want a function that finds a player by name. What if there's no match?

```
Player find_player(std::string_view name);  // ???
```

Common bad answers:

- Return a “default” `Player` — caller has no way to tell it's invalid.
- Return `Player{}` and a separate `bool` out-param — awkward signature.
- Throw — overkill for a routine miss.
- Return a `Player*` and `nullptr` — ownership / lifetime questions.

`std::optional< T >` (C++17)

A value that's either present or empty. No allocation, no pointer, no sentinel. Empty state is `std::nullopt`.

```cpp
std::optional< Player > find_player(std::string_view name);

if (auto p = find_player("Alice"); p.has_value()) {
    std::cout << p->name << '\n';
}

Player fallback = find_player("Bob").value_or(Player{"???", 0});

std::optional< int > maybe = std::nullopt;  // empty
maybe = 42;                                  // now has a value
```

A silently dropped result

The compiler is fine with this. The bug is silent.

```
std::optional< Player > find_player(std::string_view name);

find_player("Alice");  // we forgot to use the return value!
```

[[nodiscard]]

Tag the function — the compiler warns if the caller drops the result.

```
[[nodiscard]] std::optional< Player > find_player(std::string_view name);

find_player("Alice");                    // warning: discarded result
auto p = find_player("Alice");           // OK
```

[[nodiscard]]

Also useful on whole types:

```
struct [[nodiscard]] Error { /* ... */ };

Error save_file();   // every call must do something with the Error
```

What's wrong with this?

```
void greet(std::string name) {
    std::cout << "Hello, " << name << '\n';
}

greet("World");
```

Looks innocent. Yet it **allocates memory on the heap**. Every call.

Why?

The string literal `"World"` is a `const char*`. The function wants a `std::string`.

→ The compiler constructs a *temporary* `std::string`, which copies the characters into a heap buffer.

Just to print 5 characters.

“Just take it by const&!”

The reference avoids copying an existing `std::string`... but `"World"` is a `const char*`, so a **temporary `std::string` is still constructed** to bind the reference to. Same heap allocation. Just hidden better.

```cpp
void greet(const std::string& name) {
    std::cout << "Hello, " << name << '\n';
}

greet(my_string); // OK, no copy
greet("World");   // still allocates!
```

`std::string_view` (C++17)

A `string_view` is **a pointer + a length**. No ownership. No allocation.

Use it as a parameter type whenever you only need to *read* a string.

```cpp
void greet(std::string_view name) {
    std::cout << "Hello, " << name << '\n';
}

greet("World");                  // no allocation
greet(my_string);                // no copy
greet(my_string.substr(0, 5));   // still no allocation
```

Output, the painful way

`<<` chains are awkward and noisy. `printf` reads like a sentence but isn't type-safe — wrong format specifier = undefined behavior.

```
std::cout << "Player " << name
          << " has " << score
          << " points (rank #" << rank << ")\n";

printf("Player %s has %d points (rank #%d)\n",
       name.c_str(), score, rank);
```

`std::format` (C++20) / `std::print` (C++23)

Type-checked, position-checked, reads like a sentence.

```
auto line = std::format("Player {} has {} points (rank #{})\n",
                        name, score, rank);
std::cout << line;
```

C++23 cuts the indirection:

```
std::print  ("Player {} has {} points (rank #{})\n", name, score, rank);
std::println("Hello, {}!", name);  // adds the newline for you
```

References

- The Best Parts of C++ - CppCon19 - Jason Turner
- From CVS to Git: thirty years of source control

Conclusion

Now we are C++ experts!!!